In listing 3.13, `find_entry()` uses an instance of `boost::shared_lock<>` to protect it for shared, read-only access ❶; multiple threads can therefore call `find_entry()` simultaneously without problems. On the other hand, `update_or_add_entry()` uses an instance of `std::lock_guard<>` to provide exclusive access while the table is updated ❷; not only are other threads prevented from doing updates in a call `update_or_add_entry()`, but threads that call `find_entry()` are blocked too.

3.3.3. Recursive locking

With `std::mutex`, it's an error for a thread to try to lock a mutex it already owns, and attempting to do so will result in *undefined behavior*. However, in some circumstances it would be desirable for a thread to reacquire the same mutex several times without having first released it. For this purpose, the C++ Standard Library provides `std::recursive_mutex`. It works just like `std::mutex`, except that you can acquire multiple locks on a single instance from the same thread. You must release all your locks before the mutex can be locked by another thread, so if you call `lock()` three times, you must also call `unlock()` three times. Correct use of `std::lock_guard<std::recursive_mutex>` and `std::unique_lock<std::recursive_mutex>` will handle this for you.

Most of the time, if you think you want a recursive mutex, you probably need to change your design instead. A common use of recursive mutexes is where a class is designed to be accessible from multiple threads concurrently, so it has a mutex protecting the member data. Each public member function locks the mutex, does the work, and then unlocks the mutex. However, sometimes it's desirable for one public member function to call another as part of its operation. In this case, the second member function will also try to lock the mutex, thus leading to undefined behavior. The quick-and-dirty solution is to change the mutex to a recursive mutex. This will allow the mutex lock in the second member function to succeed and the function to proceed.